ASSIGNMENT 16

Android Application Security Analysis Using JD-GUI

1. Introduction

Android applications may contain sensitive information or insecure coding patterns in their compiled classes. JD-GUI is a Java decompiler that displays readable Java-like code from Java class files and supported JAR archives. By examining an authorized application's decompiled classes and methods, a security analyst can look for hardcoded credentials, insecure data handling, unsafe network settings, and other potential weaknesses.

2. Objectives

- Open an authorized application's JAR or class files in JD-GUI.
- Inspect Java classes and methods to understand their purpose.
- Identify possible hardcoded secrets and insecure coding practices.
- Record findings, assess their potential impact, and recommend suitable mitigations.

3. Requirements

- A computer with JD-GUI installed.
- A JAR file produced from an authorized APK, for example using Dex2Jar, or Java class files supported by JD-GUI.
- An APK that you own or have explicit permission to assess.
- A text editor or report document to record observations.

Use static analysis for this exercise. Do not run unknown applications, attempt to use discovered credentials, or access systems without permission.

4. Theory

JD-GUI reconstructs a readable representation of compiled Java bytecode. The displayed code is an approximation and may differ from the original source, especially when an application is obfuscated or uses compiler-generated code. Static review can reveal suspicious patterns, but a finding should be validated in context before it is reported as a confirmed vulnerability.

Hardcoded secrets may include API keys, passwords, tokens, private endpoints, or cryptographic keys embedded directly in code. Insecure practices can include storing sensitive data in plain text, disabling certificate validation, using weak cryptography, logging confidential values, or relying only on client-side checks for authorization.

5. Procedure

Step 1: Prepare the application files

Select an APK you own or have permission to examine. If starting from an APK, use the JAR produced by the previous DEX-to-JAR practical, or another authorized JAR containing the application's classes. Keep an unchanged copy of the original file.

Step 2: Open the JAR in JD-GUI

Launch JD-GUI and select File > Open File. Choose the JAR archive. The left panel displays packages and classes; selecting a class displays its decompiled Java-like code in the main panel.

Step 3: Explore classes and methods

Inspect classes with names related to configuration, authentication, networking, database access, logging, cryptography, or user preferences. Read methods that handle login, API requests, storage, encryption, and access checks. Record class and method names relevant to any suspicious behavior.

Step 4: Search for hardcoded sensitive information

Look for literal strings and configuration values that appear to be secrets, such as API tokens, passwords, private keys, or internal service URLs. Do not copy real secrets into a report or share them publicly; record the location and redact the value, for example: API_KEY = [REDACTED]. Some API keys are intentionally public identifiers, so their sensitivity and restrictions must be verified.

Step 5: Identify insecure coding practices

- Network security: check for HTTP URLs, trust-all certificate logic, or disabled hostname verification.
- Data storage: check whether passwords, tokens, or personal information are written to plain-text files, logs, or preferences.
- Cryptography: note hardcoded encryption keys, obsolete algorithms, weak random-number generation, or unsafe modes.
- Input handling: look for SQL queries built by string concatenation or insufficient validation of untrusted input.
- Authorization: determine whether important access decisions appear to rely only on client-side checks.
- Logging and errors: check whether credentials, tokens, or sensitive personal data may be written to logs or exposed in error messages.

Step 6: Assess each potential vulnerability

For each observation, record the class and method, describe the code pattern without exposing secrets, explain the possible impact, and assign a risk level based on context. Consider whether the value is actually secret, whether the vulnerable code path is reachable, what protections are present, and what access an attacker would need. Do not attempt to use credentials or test external systems without explicit authorization.

Step 7: Recommend mitigations

- Move server-side secrets out of the application package and store them in a secure backend or secret-management service.
- Rotate any exposed credential and restrict its permissions; do not rely on deleting it from the latest source alone.
- Use HTTPS with normal certificate and hostname validation.
- Store sensitive data using appropriate platform-provided secure storage and minimize retention.
- Use modern, well-reviewed cryptographic APIs and avoid embedding secret keys in client code.
- Use parameterized database queries and validate untrusted input.
- Enforce authorization on the server, not solely in the mobile client.
- Redact sensitive values from logs and error messages.

6. Example Findings

The following examples are illustrative patterns only. They are not claims about the application being examined.

Potential finding	**Possible risk**	**Risk level***	**Recommended mitigation**
Hardcoded reusable server credential	Unauthorized access if the secret is valid and exposed	High	Revoke/rotate; move secret to backend
HTTP used for sensitive traffic	Traffic may be intercepted or modified	High/Medium	Enforce HTTPS and validate certificates
Sensitive data logged	Data disclosure through device or diagnostic logs	Medium	Remove or redact sensitive logs
Client-only authorization check	Checks may be bypassed by a modified client	Medium/High	Enforce access control on the server

7. Observation Table

Class / Method	**Observed pattern**	**Risk**	**Suggested mitigation**
ConfigManager.getApiKey()	API key stored as a hardcoded string (illustrative)	High if valid and privileged	Revoke/rotate the key; keep secrets on a secure backend

NetworkClient.createConnection()	Sensitive request configured with HTTP (illustrative)	High/Medium depending on data	Use HTTPS and retain certificate/hostname validation
LoginActivity.onLogin()	Password or token may be written to logs (illustrative)	Medium	Remove sensitive logging and redact diagnostic output
DatabaseHelper.queryUser()	SQL query may be built by string concatenation (illustrative)	Medium/High depending on reachability	Use parameterized queries and validate input

8. Result

The classes and methods were reviewed using JD-GUI to identify potential hardcoded secrets and insecure coding patterns. The observation table contains illustrative sample findings to demonstrate how results should be recorded; verify each item against the authorized application actually analyzed before treating it as a real finding.

9. Precautions

- Analyze only applications for which you have permission.
- Do not use, publish, or include unredacted secrets discovered in code.
- Do not connect to external systems or test credentials without explicit authorization.
- Treat decompiled code as an approximation and verify suspicious patterns in context.
- A suspected issue should be reported as potential until sufficient evidence confirms its impact.

10. Conclusion

JD-GUI makes compiled Java classes easier to review by presenting a readable approximation of their code. Inspecting classes and methods can help identify embedded secrets, unsafe data handling, weak network protections, and other possible vulnerabilities. Responsible static analysis records the evidence, redacts sensitive values, assesses risk carefully, and recommends practical fixes.

11. References

- JD-GUI project documentation and usage instructions.
- Android Developers security best practices documentation.
- OWASP Mobile Application Security Testing Guide (MASTG).